



Figure 1: Nam for ns-2



Figure 2: The emblem of ns-3

importance of ns-3 is based on the fact that roughly one-third of all the projects done by computer science postgraduate students in India involve ns-2. This clearly tells us that ns-2 is quite popular with the academic community in our country. If ns-2 is so heavily used within academia, then why switch to ns-3? Well, papers with results depending on data obtained from the ns-2 simulator are no longer accepted by many prestigious journals for publication. Please refer to the Wikipedia article on network simulator (ns) to confirm this fact, at [http://en.wikipedia.org/wiki/Ns_\(simulator\)#ns-3](http://en.wikipedia.org/wiki/Ns_(simulator)#ns-3).

In spite of this, Indian students and researchers are still dependent on ns-2 for their data and results. And what is the condition of research publications in India? A quick search on the Internet tells us that the USA produces roughly 10 times more research papers than India. China's output in research papers is roughly four times that of India. This tells us that the use of tools like ns-2, in fact, hampers the growth of research and research publication in India. Research publication is one of the indicators of a country's development. So it is high time to migrate to advanced tools like ns-3 to increase our research output.

Another reason for the low adoption rate of ns-3 is that the research community often fails to understand that ns-3 is not backward-compatible with ns-2. ns-3 is an altogether new simulator, which is in no way related to ns-2. To my horror, even those claiming to be well versed in ns-2 have suggested that ns-1, ns-2, ns-3, and even ns-4 are no different at all. Frankly, there is no ns-4, and ns-1 and ns-2 are as different from ns-3 as chalk and cheese. This series on ns-3 aims to correct this fallacy once and for all.

A comparison of ns-2 and ns-3

ns-2 is network simulator version 2 and ns-3 is network simulator version 3. This wordplay somehow suggests that ns-3 is merely the next version of ns-2, a very common

misconception held by many. There is nothing in common between ns-2 and ns-3 except that both are network simulators. ns-2 can act only as a simulator, whereas ns-3 has the ability to act as a simulator as well as an emulator. One tough decision made by the ns-3 development team was not to make ns-3 backward compatible with ns-2. If ns-3 was backward compatible with ns-2, it might have become more popular. But then the main advantage of ns-3 is that with freshly written code it has been possible to create a new core architecture capable of supporting long term development. So I believe the decision to scrap backward compatibility with ns-2 will benefit ns-3 in the long run.

Even though the cores of both ns-2 and ns-3 are in C++, the ns-2 source code didn't have much impact on the development of the ns-3 source code. Another implementation level difference between ns-2 and ns-3 is regarding the binding language. The binding language (scripting language) associated with ns-2 is OTcl, whereas for ns-3, it is Python. With today's faster computers, it is not mandatory to have a separate scripting language to save compilation time. So in ns-3, the use of Python scripting is limited or can be avoided altogether. The dependence on a bi-language platform is another difficulty while learning ns-2, because half your time is spent on understanding the binding between the compiled hierarchy (C++ code) and the interpreted hierarchy (OTcl code). This is no longer a problem with ns-3 because C++ alone is sufficient to simulate different networking scenarios.

Other than the implementation level differences, ns-2 and ns-3 also differ in terms of the network services provided, network devices supported, network protocols and networking scenarios that can be simulated, etc. Some of the models supported by ns-2 are not supported by ns-3 and vice versa. This is one area where ns-2 outperforms ns-3. The code base of ns-2 is still larger when compared with that of ns-3. For example, the wireless sensor networks supported by ns-2 are not yet implemented in ns-3. This drawback (the smaller code base of ns-3) is getting rectified as the source code of ns-3 increases in size, day by day. Another area where ns-2 has an advantage is regarding the large chunk of contributed code provided by third party developers. ns-3 is yet to gain such massive support from third party developers.

Another area where ns-2 and ns-3 differ is in data analysis, which is done with the help of a trace file. The trace file formats are different for ns-2 and ns-3. To view the topology of the simulated network, both ns-2 and ns-3 use animators. Animators show the node alignment as well as packet movement. ns-2 uses its own animator called *Network Animator (Nam)*, which is a Tcl/Tk based animation tool and supports topology layout and packet level animation. Figure 1 shows the Nam window for an ns-2 simulation with six nodes. Packet movement through the network is also shown in Nam. Circles in the figure are nodes, and packets are shown in red colour. ns-3 does not have its own animator to display network topology and packet movement; instead, it uses an animator